CSCE883 Machine Learning **Name:** *Taylor Belcher*
Project Proposal
10/18/2026

My research area is in formal verification methods with functional programming languages and quantum computing. In particular, I am interested in formalizing proofs from mathematics. The explosion of LLMs has also led to advances in capability in this area as models have medaled in the International Mathematics Olympiad. OpenAI recently announced a solution to a Millenium Prize problem (the Navier-Stokes conjecture) and also caused a stir in the mathematical community with the release of hundreds of proofs on many topics. These natural language proofs were released along with a formalization of the results, written the interactive theorem proving language Lean. I have been working with a language called Agda, but they are related. Formalization of natural language proofs predates the recent explosion in AI capability, but they are mutually helpful. LLM's ability to generate code means we can more quickly produce formalized mathematics (Rammal et al., 2026), and the strict type requirements of formalization means we have good ways of building objective functions for reinforcement learning for AI models.

Proof formalization as a field of computer science comes from a desire in the field of mathematics to make sure that our mathematics is correct. Even before the recent release of hundreds of results from OpenAI, understanding all of published mathematics and being confident that it was all correct was already a well-known problem. Formalized proofs in Lean (or another language) are meant to be a solution to this issue. Instead of trusting that other mathematicians and their peer-reviewers to do correct work, you only have to trust the kernel of the Lean programming language to not allow a proof of true=false. Although you are ultimately just moving where you put your trust, there is good reason to trust the kernel of Lean or another language like the one I work in, Agda.

However, there is a meta-verification problem. How can you be sure that the formalization of a theorem and its proof in Lean is a faithful translation of the natural language proof? Now you not only have to read the natural language proof, but you also digest and verify the Lean code, which could be even longer and much more difficult to read since it is optimized for verification. (Although interactive theorem provers have long been working to improve on the balancing act between readability and formal correctness, like other normal programming languages.) A paper published in response to Open-AI's Navier-Stokes solution describes this problem exactly and gives examples from OpenAI's paper that identify where the Lean proof does not actually correspond to the natural language proof (Bastounis et al., 2026). It may be that the natural language proof is correct, but for the quantity of proofs and their length, we are not as justified in simply trusting the Lean certificate as we might hope.

My proposal is to build a model that is rewarded for checking correspondence between formalized mathematics and natural language mathematics rather than generating either, although I am not sure if it will end up having to be able to generate expected translations in order to check faithful translations. My idea is that while we can reasonably trust the kernel of a language like Lean to faithfully judge whether a formalized proof is correct, we will need a separate judge to give us a confidence level on how well the formalized proof corresponds to the natural language proof. In terms of "is the theorem correct", this does not matter as much because if we trust the formal proof then we have evidence the theorem is correct, but we are also concerned

about process and understanding in mathematics and we want to know what kinds of *arguments* are useful, as we have seen that understanding arguments for one proof can be fruitful for other proofs. (Whether this becomes moot as mathematics stops being a human endeavor, I am unsure. But for now I am assuming that humans will still want to pursue frontier mathematics and understand, even if AI is better at it, much like people did not stop playing chess after Stockfish.)

Lean has become the most popular language for formalizing mathematics and there are data sets with pairs of natural language and Lean code available (Compfiles, n.d.; Grammatical Framework, n.d.). MathLib on Lean has a repository of theorems (Lean Prover Community, n.d.a,n), there are the formalized IMO problems (Compfiles, n.d.), and I plan to to search for more on hugging face and github. I can use some of the OpenAI announced solutions and other available Lean formalizations as testing data. I plan to start with a pretrained transformer model such as Qwen2.5-Coder-1.5B (Qwen Team, n.d.) that takes a natural language proof and its Lean formalization together as input, with a binary classification head that judges whether they are faithful translations. I plan to use LoRA (low-rank adaptation) to fine-tune a small set of additional parameters while keeping the pretrained model weights fixed (Hu et al., 2022). I will train it on pairs that I have labeled as corresponding or not corresponding, including examples where the formalization changes an assumption, a conclusion, or a step in the argument. I will evaluate the model on how well it is able to identify corresponding pairs of natural language proofs and formalized proofs that are faithful translations, using hand-labeled examples that were not used in training. I plan to use precision, recall, and F1 for identifying unfaithful translations, along with a confusion matrix and examples of the kinds of mismatches the model catches or misses. In particular, I am interested in how often it accepts a formalization that passes Lean's checks but does not faithfully represent the natural language proof, since this is the problem that motivates the project.

As future work, perhaps this model could be put into a framework where an LLM generates candidate Lean code from a natural language proof. The candidate would have to pass Lean's checks and also be judged a faithful translation by the classification model before being accepted. If it failed either check, the LLM could try another candidate. This would give us a way to use both formal correctness and correspondence with the original argument when choosing a formalization, although the classification model would still be a learned judge and would not give the same kind of guarantee as the Lean kernel. For this project, I plan to focus on building and evaluating the judge, but I am interested in whether it could eventually help guide the generation of formalizations as well. I will start with Lean, but I am also interested in building something for Agda. Agda is far less popular and there are not nearly as many resources for data to train on. I may have to generate some data with LLMs and hand-label, but that will result in small datasets. Perhaps it would be possible to train on Lean and then use that model to specialize to Agda.

References

Ahmad Rammal, Niket Patel, Fabian Gloeckle, Amaury Hayat, Julia Kempe, Remi Munos, Charles Arnal, and Vivien Cabannes. *Formalizing Mathematics at Scale*. 2026. https://arxiv.org/abs/2605.29955.

Alexander Bastounis, Fabian Circelli, and Anders C. Hansen. *Navier-Stokes lost in translation: Why Lean verification of AI autoformalisation does not guarantee correct natural language proofs*. 2026. https://arxiv.org/abs/2610.08144.

Compfiles. *Catalog of Math Problems Formalized in Lean: IMO problems*. https://dwrensha.github.io/compfiles/imo.html. Accessed October 13, 2026.

Grammatical Framework. *Informath: Mathematical terms, definitions, and propositions in as many languages as possible*. https://github.com/GrammaticalFramework/informath. Accessed October 13, 2026.

Lean Prover Community. *mathlib4: The math library of Lean 4*. https://github.com/leanprover-community/mathlib4. Accessed October 13, 2026.

Lean Prover Community. *1000+ theorems in Lean*. https://leanprover-community.github.io/1000.html. Accessed October 13, 2026.

Qwen Team. *Qwen2.5-Coder-1.5B model card*. https://huggingface.co/Qwen/Qwen2.5-Coder-1.5B. Accessed October 13, 2026.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. *LoRA: Low-Rank Adaptation of Large Language Models*. International Conference on Learning Representations, 2022. https://arxiv.org/abs/2106.09685.